universität innsbruck
Department of Computer Science

Recommender Systems

Collaborative Filtering

Eva Zangerle

Collaborative Filtering (CF)

- Core idea: recommend items based on preferences of other users.
- Main underlying assumption: users that had similar taste in the past, will have similar taste in the future.
- Collective behavior as strong signal for recommendation: what users watched, clicked, rated, etc.
- Collaborative Filtering uses community data:
  - Collect user feedback given ratings of items (explicit or implicit)
  - Use ratings of all or some users collectively to determine whether to recommend a new item to a user.
- Most widely used approach for recommendation.
- Applicable to many domains (no explicit domain knowledge necessary).

User-Item Matrix

- Ratings are represented in a user-item matrix:
  - Rows: users
  - Columns: items
  - Entries: ratings or interactions
- Missing entries correspond to unknown preferences; the goal of CF is to predict these missing entries.

	Indiana Jones	Microcosmos	Avengers	Pete's Dragon
Sara	4	5		
Jesper	4		5	
Therese	5			3
Helle	4			5
Pietro		3	4	3
Ekaterina	3		3	0

Example taken from Practical Recommender Systems: Kim Falk; Manning Publications, 2019.

Two CF Paradigms: Memory- vs. Model-based

- Memory-based (neighborhood) CF
  - No model is built, but predictions are made on the fly.
  - Nearest neighbor approach (find most similar users or items).
  - Simple and well-established method.
  - Results easy to interpret for transparency.
- Model-based CF
  - User-item matrix is factorized, ratings are explained by 2”latent factors” in a low-dimensional space.
  - Users and items can be represented in the same space.
  - Once model is created, rating prediction is very fast.
  - Well-established method.
  - Latent factors may not be easy to interpret (just describe variance in the data).

Memory-based (Neighborhood) CF

CF: User- vs. Item-Based

- User-based CF
  - Find similar users based on rated items (vectors over items).
  - Predict rating as weighted combination of most similar users' ratings.
- Item-based CF
  - Find similar items based on user ratings (vectors over users).
  - Predict rating as weighted combination of most similar items' ratings.
- In real-world applications, item-based methods can scale better.
- Rating biases should be accounted for.

User-Based CF

- Goal: predict how user u would rate item i.
- Idea: Identify similar users, use their ratings to predict missing rating.
  - If Alice and Bob rated many movies similarly, Bob's opinion may help predict what Alice likes next.
- Algorithm outline:
  - Compute similarity of target user to all users that have rated the item to predict.
  - Select k most similar users (k-neighborhood).
  - Compute prediction for item from a weighted combination of the item's ratings of users in neighborhood (weights correspond to similarity).
    - Weights are derived from similarity scores.
    - Only neighbors who rated the target item are considered for prediction.

User-Based CF: Similarity

- Calculate similarity of active user a to all users u that have rated item i to predict.
- Commonly used similarity measure is Pearson's correlation coefficient:

$$sim(a,u) = \frac{\sum_{j \in I_{a,u}} (r_{a,j} - \bar{r}_a)(r_{u,j} - \bar{r}_u)}{\sqrt{\sum_{j \in I_{a,u}} (r_{a,j} - \bar{r}_a)^2} \sqrt{\sum_{j \in I_{a,u}} (r_{u,j} - \bar{r}_u)^2}}$$

where $I_{a,u}$ is the set of items co-rated by users a and u, $r_{a,j}$ is the rating of user a for item j, and $\bar{r}_x$ is the average rating of a given user x.

- Accounts for user rating biases by centering ratings around the user mean (different absolute ratings may differ, but relative order is preserved).
- Coefficient is in [-1, 1].
- Pearson correlation requires variance in user ratings, else it is undefined.

User-Based CF: Similarity cont.

- Pearson's correlation has shown to work best for this purpose.
- Potential alternatives:
  - (adjusted) cosine similarity (see later),
  - Spearman rank correlation (e.g., to deal with different numeric ranges of ratings)
  - Kendall's τ correlation
  - Mean squared differences
  - Entropy

User-Based CF: Neighborhood Selection

- Select k users that have highest similarity (neighborhood).
- Predefine k, sort according to similarity scores, and select k highest.
- Choice of k dependent on dataset, but typically small values of k (e.g., 20-50) work best in practice ➜ tuned on validation set.
- Tradeoffs:
  - Small k: more personalized, but less coverage and more noise (unreliable similarity estimates).
  - Large k: more coverage, more robust, but less personalized and more computationally expensive.

User-Based CF: Rating Prediction

- Compute prediction for item i from a weighted combination of the ratings of users in neighborhood.
- Predict rating $\hat{r}_{a,i}$ of active user a for item i as:

$$\hat{r}_{a,i} = \bar{r}_a + \frac{\sum_{u \in N(a)} sim(a,u) \cdot (r_{u,i} - \bar{r}_u)}{\sum_{u \in N(a)} |sim(a,u)|}$$

  where $N(a)$ is the set of k most similar users to user a who have rated item i, $sim(a,u)$ is the similarity between user a and user u, $r_{u,i}$ is the rating of user u for item i, and $\bar{r}_a$ and $\bar{r}_u$ are the average ratings of users a and u, respectively.
- Starts from a's rating bias and adds deviations based on similarity.
- After predicting all missing values of a, the items with highest predicted ratings can be recommended.

User-Based CF: Example

	Item 1	Item 2	Item 3	Item 4	Item 5
User 1	3		2	3	3
User 2	4	3	4	3	
User 3	3	2	1	4	4
User 4		5	4	3	1
User a	5		3	4	?

User-Based CF: Example cont.

	Item 1	Item 2	Item 3	Item 4	Item 5
User 1	3		2	3	3
User 2	4	3	4	3	
User 3	3	2	1	4	4
User 4		5	4	3	1
User a	5		3	4	?

- User 2 has not rated Item 5, ignore.
- Step 1: Calculate similarities (Pearson correlation)

$$\text{sim}(a, u_1) = \frac{(5-4)(3-2.67) + (3-4)(2-2.67) + (4-4)(3-2.67)}{\sqrt{(5-4)^2 + (3-4)^2 + (4-4)^2} \cdot \sqrt{(3-2.67)^2 + (2-2.67)^2 + (3-2.67)^2}} = \frac{0.33 + 0.67 + 0}{\sqrt{2} \cdot \sqrt{0.11 + 0.44 + 0.11}} = \frac{1}{1.15} = 0.87$$

$$\text{sim}(a, u_3) = \frac{(5-4)(3-2.67) + (3-4)(1-2.67) + (4-4)(4-2.67)}{\sqrt{(5-4)^2 + (3-4)^2 + (4-4)^2} \cdot \sqrt{(3-2.67)^2 + (1-2.67)^2 + (4-2.67)^2}} = \frac{0.33 + 1.67 + 0}{\sqrt{2} \cdot \sqrt{4.66}} = \frac{2}{3.05} = 0.65$$

$$\text{sim}(a, u_4) = \frac{(3-3.5)(4-3.5) + (4-3.5)(3-3.5)}{\sqrt{(3-3.5)^2 + (4-3.5)^2} \cdot \sqrt{(4-3.5)^2 + (3-3.5)^2}} = \frac{-0.25 - 0.25}{0.5} = -1$$

We will not consider uncorrelated users or users with negative correlation.

User-Based CF: Example cont.

- Sort and select neighbors (for the setting k = 2), N = {User 1, User 3}
- Calculate prediction for Item 5 for user a:

$$\hat{r}_{a,5} = \bar{r}_a + \frac{sim(a,u_1)\cdot(r_{u_1,5} - \bar{r}_{u_1}) + sim(a,u_3)\cdot(r_{u_3,5} - \bar{r}_{u_3})}{|sim(a,u_1)| + |sim(a,u_3)|} = 4 + \frac{0.87\cdot(3-2.67) + 0.65\cdot(4-2.67)}{0.87+0.65} = 4 + \frac{0.29+0.87}{1.52} = 4 + 0.76 = 4.76$$

- Thus, we predict a rating of 4.76 for Item 5 for user a, and would recommend this item to the user.
- NB: Rating prediction has to be done for all items to recommend the top-k items with highest predicted ratings.

User-Based CF: Advantages

- Simple and well-established method.
- Results easy to interpret for transparency (e.g., explainable recommendations by showing similar users and their ratings).
- No explicit domain knowledge necessary, thus applicable to many domains.
- Can be used for both rating prediction and top-k recommendation (by recommending items with highest predicted ratings).

User-Based CF: Practical Issues

- Small overlap in co-rated items: similarity estimates may be unreliable.
- Choice of neighborhood size k (coverage vs. noise dilemma)
  - Small k: unstable, noisy
  - Large k: more coverage, but less personalization
- Popular items may dominate.
- Negative correlations and uncorrelated users are often ignored in practice.
- Cold-start problem: no ratings for new users/items, thus no recommendations possible.

User-Based CF: Extensions

- Users' agreement on items that are controversial should have more impact than agreement on items everybody likes
  - Inverse user frequency: items liked by all get less influence
  - Variance weighting factor: items with high variance in ratings get more influence
- Neighborhood selection should also consider how many items have been co-rated (small overlap might lead to high correlation by chance)
  - Significance weighting: devalue weights if less than 50 co-rated items
  - Default voting: assume default values for missing ratings to calculate correlations on union of rated items rather than on intersection

Item-Based CF

- Goal: predict how user u would rate item i via similar items.
- Idea: identify similar items based on all their user ratings; use ratings active user gave to those similar items to predict missing rating
- Algorithm very similar to user-based CF recommendation (instead of the rows in the rating matrix, we now look at the columns):
  - Compute similarity of items by comparing rating vectors (over users)
  - Select k items that have highest similarity (k-neighborhood)
  - Compute prediction for item from a weighted combination of the ratings of items in neighborhood

Item-Based CF: Similarity

- Item similarity predominantly computed via cosine similarity:

$$sim(i,j) = \frac{\sum_{u \in U_{i,j}} r_{u,i} \cdot r_{u,j}}{\sqrt{\sum_{u \in U_{i,j}} r_{u,i}^2} \cdot \sqrt{\sum_{u \in U_{i,j}} r_{u,j}^2}}$$

  where $U_{i,j}$ is the set of users who have rated both items i and j, and $r_{u,i}$ is the rating of user u for item i.
- Cosine similarity is in [0, 1], if all ratings are positive (e.g., 1-5 stars).
- Cannot deal with missing ratings, thus only users that have rated both items are considered for similarity calculation.
- Does account for different item popularities by normalizing by the magnitude of the rating vectors.
- Does not account for user rating biases.

Item-Based CF: Similarity cont.

- Adjusted cosine similarity accounts for user rating biases by centering ratings around the user mean:

$$sim(i,j) = \frac{\sum_{u \in U_{i,j}} (r_{u,i} - \bar{r}_u) \cdot (r_{u,j} - \bar{r}_u)}{\sqrt{\sum_{u \in U_{i,j}} (r_{u,i} - \bar{r}_u)^2} \cdot \sqrt{\sum_{u \in U_{i,j}} (r_{u,j} - \bar{r}_u)^2}}$$

  where $\bar{r}_u$ is the average rating of user u.
- Subtracts user u's rating bias from the ratings of items i and j, thus accounts for different user rating biases.
- Adjusted cosine similarity is in [-1, 1].

Item-Based CF: Rating Prediction

- Predict rating $\hat{r}_{u,i}$ of active user u for item i as:

$$\hat{r}_{u,i} = \frac{\sum_{j \in N(i)} sim(i,j) \cdot r_{u,j}}{\sum_{j \in N(i)} |sim(i,j)|}$$

where $N(i)$ is the set of k most similar items to item i that user u has rated, and $sim(i,j)$ is the similarity between items i and j.

Item-Based CF: Example

	Item 1	Item 2	Item 3	Item 4	Item 5
User 1	3		2	3	3
User 2	4	3	4	3	
User 3	3	2	1	4	4
User 4		5	4	3	1
User a	5		3	4	?

- User 2 won’t help for item 5, hence discard.
- User a has not rated Item 2, hence discard.

Item-Based CF: Example cont.

Subtract mean user rating (saves some time later):

	Item 1	Item 3	Item 4	Item 5
User 1	0.25	-0.75	0.25	0.25
User 3	0.2	-1.8	1.2	1.2
User 4		0.75	-0.25	-2.25
User a	1	-1	0	?

Step 1: Calculate adjusted cosine similarity between items:

$$sim(i_5, i_1) = \frac{0.25 \cdot 0.25 + 1.2 \cdot 0.2}{\sqrt{0.25^2 + 1.2^2} \cdot \sqrt{0.25^2 + 0.2^2}} =\approx 0.76$$

$$sim(i_5, i_3) = \frac{0.25 \cdot -0.75 + 1.2 \cdot (-1.8) + (-2.25) \cdot 0.75}{\sqrt{0.25^2 + 1.2^2 + (-2.25)^2} \cdot \sqrt{0.75^2 + (-1.8)^2 + 0.75^2}} = -0.75$$

$$sim(i_5, i_4) = \frac{0.25 \cdot 0.25 + 1.2 \cdot 1.2 + (-2.25) \cdot (-2.25)}{\sqrt{0.25^2 + 1.2^2 + (-2.25)^2} \cdot \sqrt{(-0.25)^2 + 1.2^2 + (-0.25)^2}} = 0.64$$

Item-Based CF: Example

Step 2: Select k=2 most similar items (i_1 and i_4) and calculate prediction for Item 5 for user a:

$$\hat{r}_{a,i_5} = \frac{0.76 \cdot 5 + 0.64 \cdot 4}{|0.76| + |0.64|} = 4.54$$

Item-Based CF: Advantages

- Better suited for large-scale recommenders than user-based CF.
- Preprocessing can be performed offline, i.e., all item-to-item similarities can be calculated in advance (need update after some time)
- n users and m items: in worst case $n \times m$ evaluations.
- More realistic: users rate only small number of items, but items are rated by many users ➜ items are more stable than users, thus item-based CF can be more accurate.
- For item-based CF, at runtime, recommendation in real-time possible.

Summary Memory-based CF

- User-based CF: find similar users, use their ratings to predict missing rating.
- Item-based CF: find similar items, use active user's ratings of those items to predict missing rating.
- Both use local evidence using similar users/items, and prediction via weighted averaging.
- Very strong baseline and widely used in practice, but has limitations (e.g., sparsity, cold-start problem, scalability).
- Main limitation:
  - Does not capture complex interactions between users and items (only pairwise).
  - No global patterns in the data are captured, thus cannot discover latent factors that explain user preferences and item characteristics.
  - Example: users that like action movies may also like comedy movies, but this relationship is not captured by neighborhood-based CF.

Model-based CF (Matrix Factorization)

Matrix Factorization: Idea

- Assumption: ratings are driven/can be explained by a small number of latent (hidden) factors.
- Examples:
  - Serious vs. light-hearted movies
  - Niche vs. mainstream movies
  - Action vs. romance movies
  - Modern vs. classic movies
- Goal: discover these latent factors and represent users and items in terms of these factors.

Latent Space

- Each user is represented by a vector of latent factor preferences (e.g., how much they like action, comedy, etc.).
- Each item is represented by a vector of latent factor characteristics (e.g., how much it is an action movie, comedy movie, etc.).

➜ Users and items are represented in the same latent space, thus we can compute similarity between users and items in this space to make recommendations.

Dimensionality Reduction

We aim to learn a latent space with substantially lower dimensionality than the original user-item matrix, thus we can capture the underlying structure of the data while reducing noise and improving generalization.

Latent Space Example

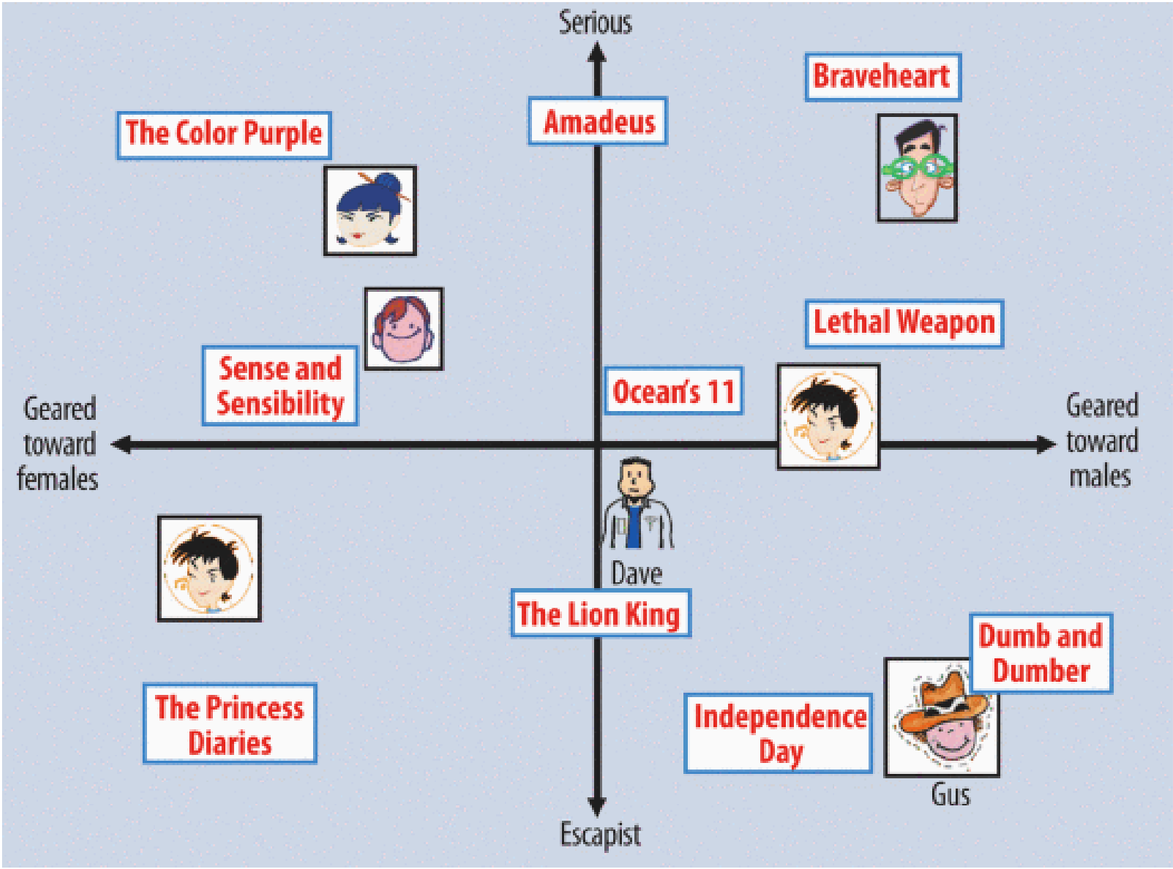


Figure taken from Y. Koren, R. Bell and C. Volinsky, Matrix Factorization Techniques for Recommender Systems, in Computer, vol. 42, no. 8, pp. 30-37, Aug. 2009, doi: 10.1109/MC.2009.263.

Latent Space: Prediction

- If user and item point in similar directions, predicted preference is high.
- Once users and items are represented in the same latent space, we can compute the predicted rating of a user for an item via their similarity in the latent space.
- Similarity measure: dot product of their latent factor vectors p_u and q_i:

$$\hat{r}_{u,i} = p_u^T q_i$$

- Dot product captures the interaction between user preferences and item characteristics in the latent space.
- Dot product is large if
  - Both vectors emphasize the same latent factors (e.g., user likes action movies and item is an action movie), and
  - Both vectors have large magnitudes (e.g., user has strong preferences and item has strong characteristics).

Latent Space: Prediction Example

User preference vector:

$$p_u = [0.9, 0.2]$$

Movie attributes vector:

$$q_i = [0.8, 0.1]$$

Prediction:

$$0.9 \cdot 0.8 + 0.2 \cdot 0.1 = 0.74$$

Latent Space: Generalization

- Even if a user has not rated an item, we can still predict their preference.
- Use the latent factor representations to make predictions.
- Example: user has not rated comedy movie, but has high value on comedy latent factor, we can recommend movies that have strong comedy characteristics (high value on comedy latent factor).
- This allows for better generalization and the ability to make recommendations for new users and items.

Matrix Factorization

$$R \approx PQ^T$$

where:

- R = rating matrix
- P = user factors
- Q = item factors
- Instead of memorizing all entries (i.e., the full user-item matrix), learn compact representations of small number of latent factors (model vs. memory).

Low-Rank Approximation

- Matrix factorization can be seen as a low-rank approximation of the user-item matrix.
- Many large matrices contain redundancy.
- A smaller number of factors can explain much of the useful signal.

➜ Reduce noise and storage requirements by only keeping the most important latent factors that capture the underlying structure of the data.

Low-Rank Approximation: SVD

- Singular Value Decomposition (SVD) is a common linear algebra method for matrix factorization.
- SVD decomposes the user-item matrix R into three matrices: $R = U\Sigma V^T$, where:
  - U is the user factor matrix (left singular vectors)
  - Σ is a diagonal matrix of singular values (importance of each latent factor)
  - V^T is the item factor matrix (right singular vectors)
- By keeping only the top k singular values and corresponding vectors, we can obtain a low-rank approximation of the original matrix.
- However, standard SVD assumes all matrix entries are observed - not applicable to sparse user-item matrices with missing values.

Matrix Factorization: Objective Function

$$R \approx PQ^T$$

- The goal of matrix factorization is to find user and item factor matrices P and Q that minimize the difference between the observed ratings in R and the predicted ratings from PQ^T.
- Hence, we learn factors that fit the known ratings with the objective function:

$$\min_{P,Q} \sum_{(u,i)\in K} (r_{u,i} - p_u^T q_i)^2$$

where K is the set of user-item pairs for which we have observed ratings, $r_{u,i}$ is the observed rating of user u for item i, p_u and q_i are the latent factor vectors for user u and item i, respectively.

Overfitting and Regularization

- If the model is too complex (e.g., too many latent factors), it may fit the training data very well but fail to generalize to new data (overfitting).
- Our model should prefer simpler explanations that capture the underlying structure of the data rather than noise ➜ regularization.
- Regularization does not directly reduce the number of dimensions k, but penalizes large values in the user and item factor matrices, thus encouraging simpler models that are less likely to overfit.
  - Extreme parameter values become costly and thereby, less likely.
  - Weakly supported latent factors are shrinked towards zero.

Overfitting and Regularization cont.

Hence, we extend our objective function with a regularization term that penalizes large values in the user and item factor matrices:

$$\min \sum_{(u,i)\in K} (r_{ui} - p_u^T q_i)^2 + \lambda(\|p_u\|^2 + \|q_i\|^2)$$

where λ controls the strength of the penalty and $\|\cdot\|$ denotes the Euclidean norm.

Choosing λ and k

- Both λ (regularization strength) and k (number of latent factors) are hyperparameters that need to be tuned on a validation set.
- Typically, small values of k (e.g., 10-100) work well in practice, but the optimal value depends on the dataset and the amount of data available.
- λ is typically chosen from a range of values (e.g., 0.01, 0.1, 1, 10) using cross-validation to find the value that minimizes the validation error.

Optimization and Learning Methods

- The objective function is non-convex, thus we cannot guarantee finding the global minimum.
- Common optimization methods include:
  - Stochastic Gradient Descent (SGD): iteratively update user and item factors based on the gradient of the loss function with respect to the observed ratings.
  - Alternating Least Squares (ALS): alternately fix one factor matrix and solve for the other using least squares optimization, which can be more efficient for large datasets.

Optimization and Learning Methods cont.

- Both methods can be parallelized and implemented efficiently for large-scale datasets.
- SGD is more commonly used in practice due to its simplicity and efficiency
  - Only parameters of one rating at a time are updated.
- Both methods can be sensitive to the choice of hyperparameters (e.g., learning rate for SGD, regularization strength).

Matrix Factorization Extension: Bias Models

- Not every rating difference reflects user-item interaction (remember adjusted similarity for neighborhood CF).
- Examples:
  - Some users may consistently give higher ratings than others (user bias).
  - Some items may receive higher ratings on average than others (item bias).
- Without correcting for these systematic biases, we waste model capacity to learn these biases.

Bias Models: Global Average

- Simplest predictor is the global average rating μ across all users and items:

$$\hat{r}_{u,i} = \mu$$

- However, this does not account for user and item biases, thus may lead to poor predictions

Bias Models: User and Item Biases

- Some users rate above or below the global average: user bias b_u.
- Some items receive higher or lower ratings than average: item bias b_i.

Bias Models: Global Baseline

$$\hat{r}_{ui} = \mu + b_u + b_i$$

- Example:
  - Mean movie rating: 3.7 stars.
  - User Alice has a bias of -0.2 stars (she tends to give lower ratings than average).
  - The Sixth Sense has a bias of 0.5 stars.
  - Baseline estimate: 3.7 (global average) - 0.2 (Alice's bias) + 0.5 (The Sixth Sense's bias) = 4.0 stars.
- This baseline model can already provide a reasonable prediction, hence, a reasonable extension to the matrix factorization model. s

Matrix Factorization with Biases

- To account for these biases, we can extend our model to include bias terms:

$$\hat{r}_{u,i} = \mu + b_u + b_i + p_u^T q_i$$

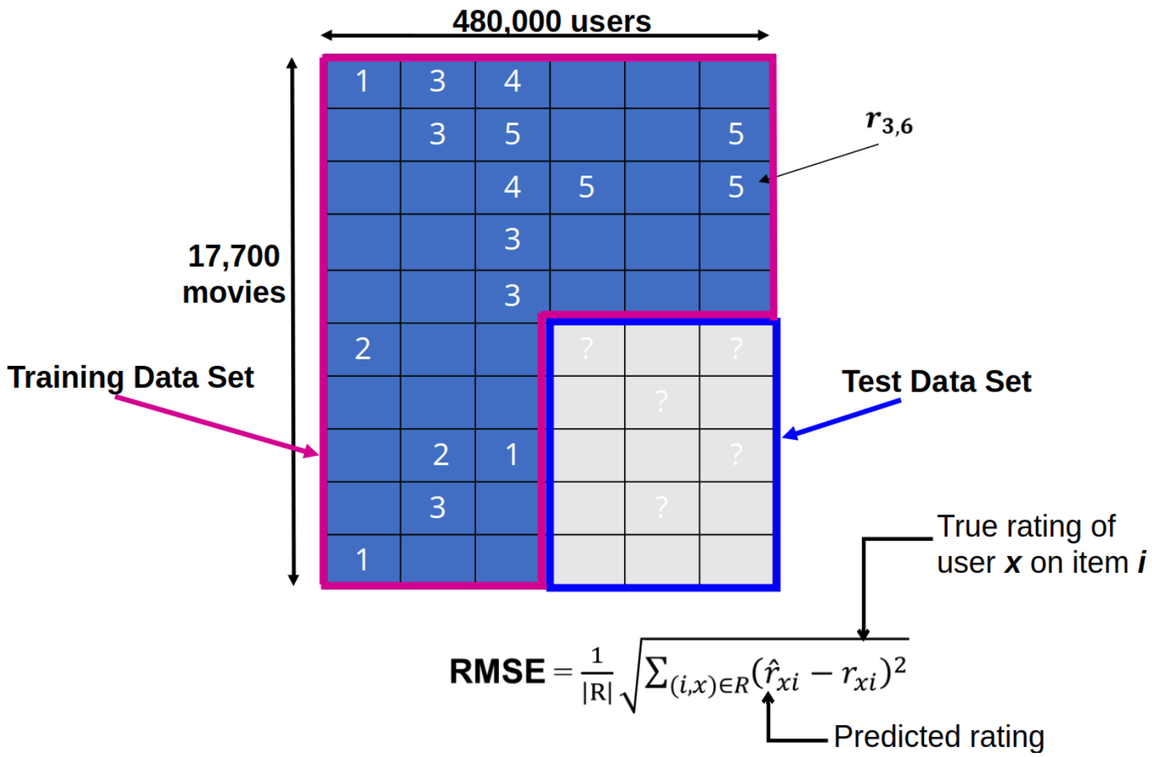
  where μ is the global average rating, b_u is the user bias, b_i is the item bias, and p_u and q_i are the user and item latent factors, respectively.
- This now combines global tendencies (biases) with personalized interactions (latent factors), thus providing a more accurate prediction of user preferences.

Netflix Prize

Historical Context

In 2006, Netflix launched a public competition: Prize: $1,000,000 for the first team to achieve a 10% improvement over Netflix's existing algorithm (Cinematch) on a test set of user ratings.

Netflix Prize: Dataset



$$\text{RMSE} = \frac{1}{|R|}\sqrt{\sum_{(i,x)\in R}(\hat{r}_{xi} - r_{xi})^2}$$

Netflix Prize: Setup

- Training set: 100 million ratings from 480,000 users on 17,000 movies.
- Test set: 2.8 million ratings (hidden from participants).
- Evaluation metric: Root Mean Squared Error (RMSE) between predicted and actual ratings in the test set.
- Baseline (Cinematch) RMSE: 0.9525
- Target RMSE for winning: 0.8572 (10% improvement over baseline).
- Leaderboard updated in real-time, showing the best RMSE achieved by any team at any point in time.
- Competition ran for 3 years, with over 40,000 teams participating.

Netflix Prize: Leaderboard

NETFLIX

Netflix Prize

COMPLETED

Home | Rules | Leaderboard | Update | Download

Leaderboard

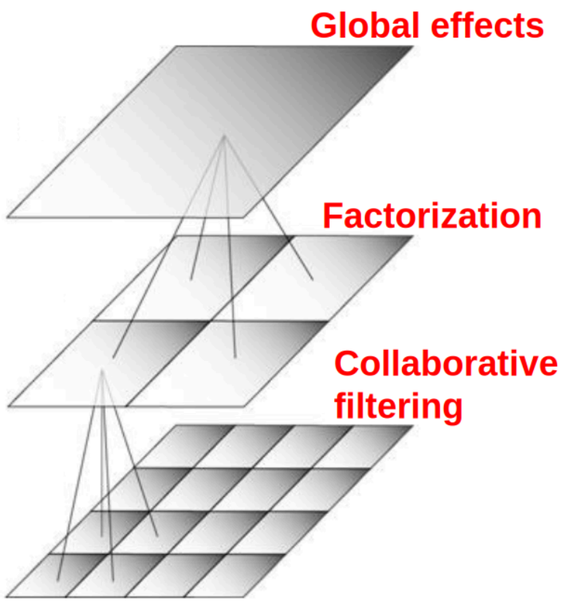
Showing Test Score. Click here to show quiz score

Display top 20 leaders.

Rank	Team Name	Best Test Score	% Improvement	Best Submit Time
Grand Prize - RMSE = 0.8567 - Winning Team: BellKor's Pragmatic Chaos				
1	BellKor's Pragmatic Chaos	0.8567	10.06	2009-07-26 18:18:28
2	The Ensemble	0.8567	10.06	2009-07-26 18:38:22
3	Grand Prize Team	0.8582	9.90	2009-07-10 21:24:40
4	Opera Solutions and Vandelay United	0.8588	9.84	2009-07-10 01:12:31
5	Vandelay Industries !	0.8591	9.81	2009-07-10 00:32:20
6	PragmaticTheory	0.8594	9.77	2009-06-24 12:06:56
7	BellKor in BigChaos	0.8601	9.70	2009-05-13 08:14:09
8	Dace_	0.8612	9.59	2009-07-24 17:18:43
9	Feeds2	0.8622	9.48	2009-07-12 13:11:51
10	BigChaos	0.8623	9.47	2009-04-07 12:33:59
11	Opera Solutions	0.8623	9.47	2009-07-24 00:34:07
12	BellKor	0.8624	9.46	2009-07-26 17:19:11
Progress Prize 2008 - RMSE = 0.8627 - Winning Team: BellKor in BigChaos				
13	xiangliang	0.8642	9.27	2009-07-15 14:53:22
14	Gravity	0.8643	9.26	2009-04-22 18:31:32
15	Ces	0.8651	9.18	2009-06-21 19:24:53
16	Invisible Ideas	0.8653	9.15	2009-07-15 15:53:04
17	Just a guy in a garage	0.8662	9.06	2009-05-24 10:02:54
18	J Dennis Su	0.8666	9.02	2009-03-07 17:16:17
19	Craig Carmichael	0.8666	9.02	2009-07-25 16:00:54
20	acmehill	0.8668	9.00	2009-03-21 16:20:50
Progress Prize 2007 - RMSE = 0.8723 - Winning Team: KorBell				

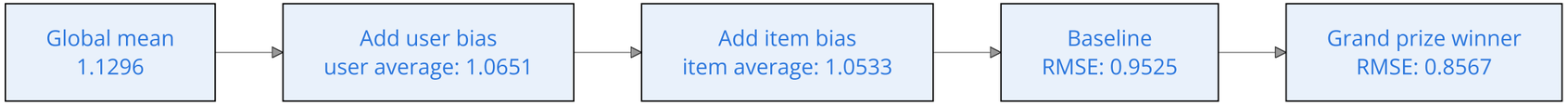
Netflix Prize: Winning Solution

- The winning team (BellKor's Pragmatic Chaos) achieved a 10.06% improvement over the baseline, with an RMSE of 0.8567.
- Approach relied on multi-scale modeling of the data:
  - Combine top level, regional modeling of the data, with refined, local view:
    - Global: Overall deviations of users/movies
    - Factorization: Addressing regional effects
    - Collaborative filtering: Extract local patterns



Netflix Prize: Global Effects Component

- Global effects: overall tendencies in the data that can be captured by simple models ➜ global average bias, user and item biases.
- Progression of RMSE improvements on the Netflix Prize leaderboard:



- Bias models alone can already provide a improvement, but personalization is still missing (e.g., users that like action movies may also like comedy movies, but this relationship is not captured by bias models).

Collaborative Filtering Component

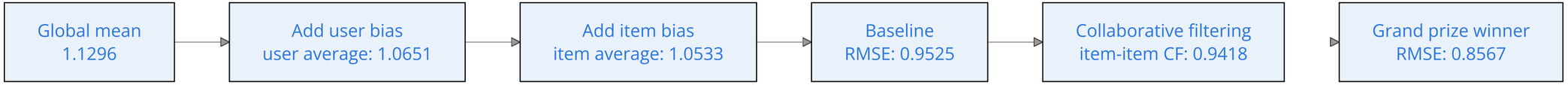
- Derive unknown ratings from ratings of similar items (item-item variant).
- Define a similarity measure s_{ij} between items i and j.
- Select the k-nearest neighbors of item i.
- Use the user's ratings on these neighbors to predict the missing rating.

$$\hat{r}_{x,i} = \frac{\sum_{j \in N(i;x)} sim_{ij}\, r_{x,j}}{\sum_{j \in N(i;x)} |sim_{ij}|}$$

where:

Collaborative Filtering Component cont.

- This approach captures local patterns in the data (e.g., users that like action movies may also like comedy movies), thus providing a more personalized recommendation.



Modeling Local & Global Effects

- In practice, we obtain better estimates if we model systematic deviations explicitly.
- Some effects are global and not purely personalized.
- We define a baseline predictor:

$$b_{x,i} = \mu + b_x + b_i$$

where:

- μ: overall mean rating (global effect)
- b_x: rating deviation of user x (user bias), defined as $b_x = \bar{r}_x - \mu$
- b_i: rating deviation of item i (item bias), defined as $b_i = \bar{r}_i - \mu$

This gives a stronger starting point than using raw neighborhood averages alone.

Why Basic Neighborhood Models Are Limited

Even with bias correction, standard similarity-based methods have limitations:

1. Finding a suitable similarity measure is difficult.
2. Pairwise item similarities neglect broader interdependencies.
3. Weighted averaging can be restrictive.
4. Similarity estimates can be noisy in sparse data.

Idea: Interpolation Weights w_{ij}

- Instead of manually defining similarities s_{ij}, we can learn weights directly from data.
- Replace: sim_{ij} with learned parameters w_{ij} in the prediction equation.
- These weights capture how strongly rating item j should influence prediction for item i.

Learned Neighborhood Prediction

Prediction now becomes:

$$\hat{r}_{x,i} = b_{x,i} + \sum_{j \in N(i;x)} w_{ij} \left(r_{x,j} - b_{x,j} \right)$$

Interpretation:

- Start with the baseline estimate.
- Add deviations from previously rated neighboring items.
- Learn the correction weights directly from data.

Recommendations via Optimization

- We can view the recommendation problem as an optimization problem where we want to find the model parameters (e.g., user and item factors, bias terms, interpolation weights) that best fit the observed ratings in the training data.
- To quantify the quality of each observed rating, we define a loss function based on RMSE, using as inputs:
  - Predicted rating: $\hat{r}_{x,i}$
  - True rating: $r_{x,i}$

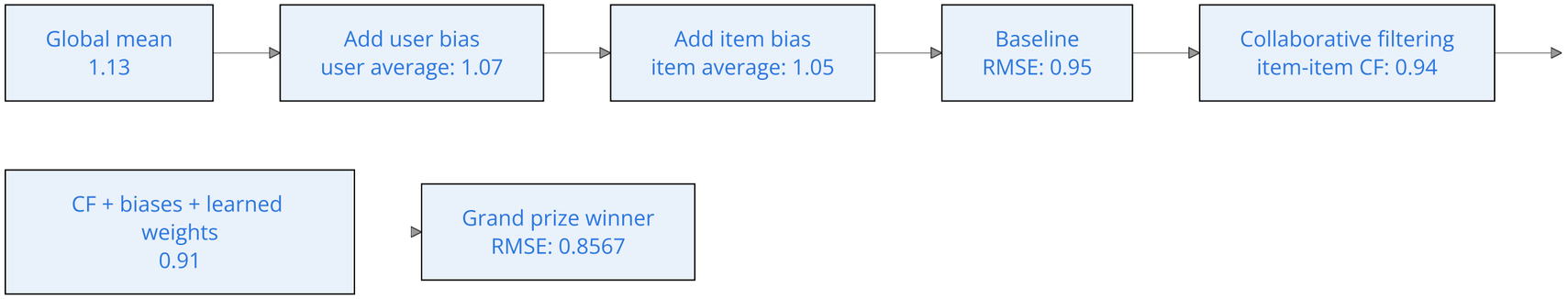
The model parameters are chosen to minimize the difference.

Evaluation Setup

We cannot train directly on unknown future ratings, instead we

1. Learn from known ratings.
2. Validate on held-out data.
3. Use the trained model to predict missing ratings.

This optimization view became central in modern recommender systems.



Overfitting and Regularization

- To solve overfitting, we introduce regularization:
  - Allow rich model where there are sufficient data.
  - Shrink aggressively where data are scarce (simple model, avoid overfitting).

$$\min_{P,Q} \sum_{\text{training}} (r_{xi} - q_i p_x)^2 \; + \; \lambda \left[\sum_x \|p_x\|^2 + \sum_i \|q_i\|^2 \right]$$

Latent Factors

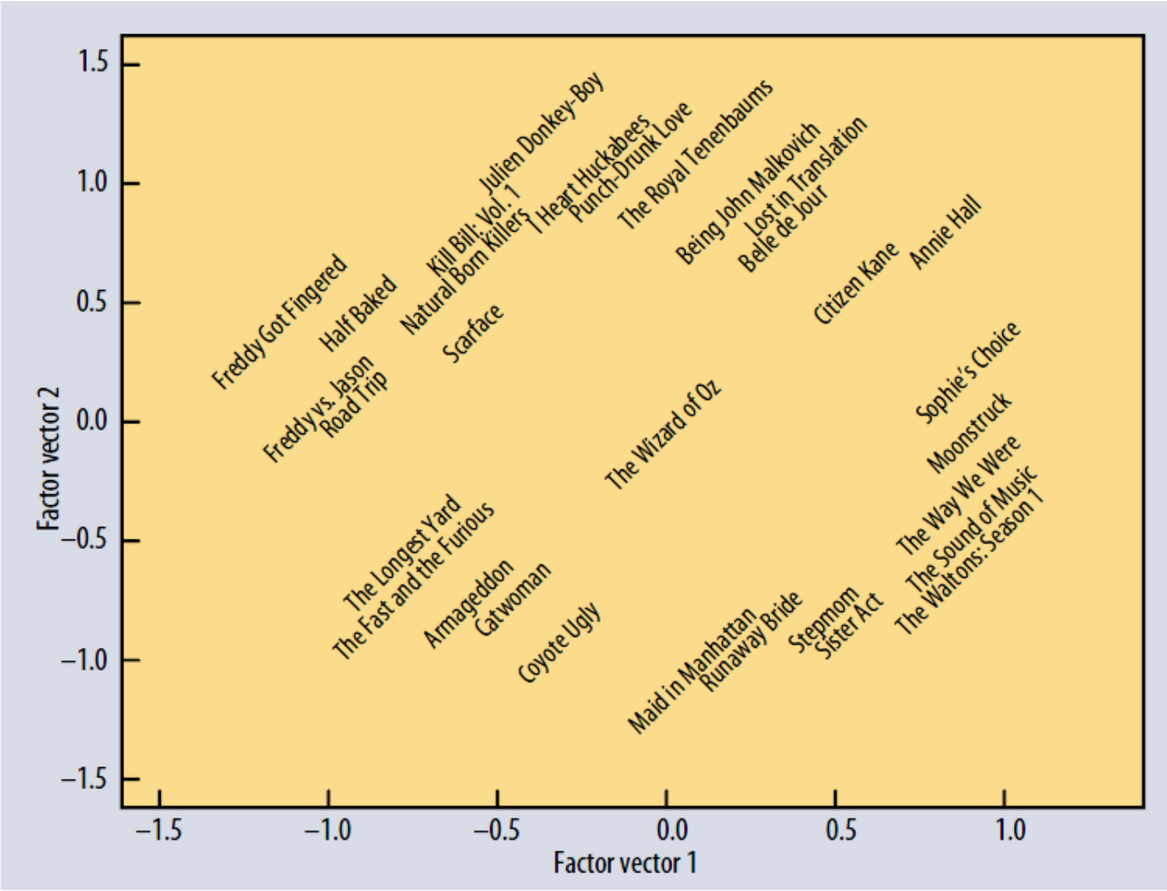
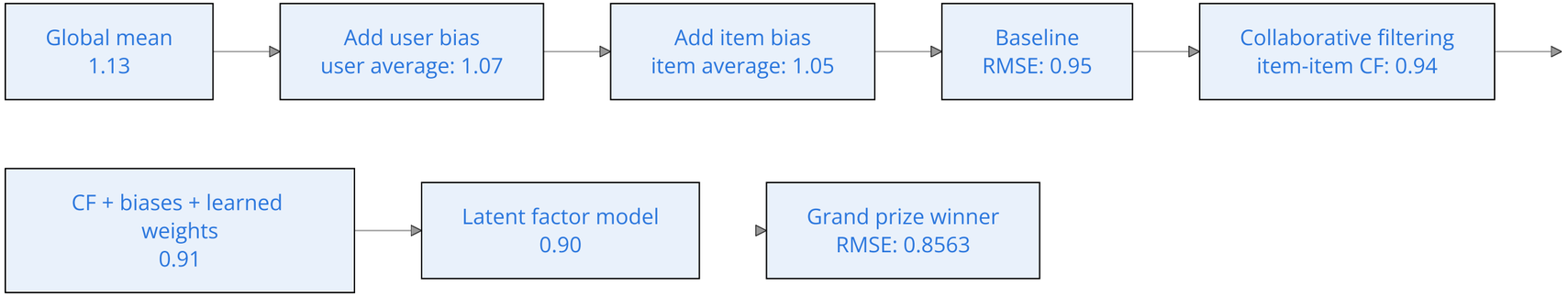


Figure taken from Y. Koren, R. Bell and C. Volinsky, Matrix Factorization Techniques for Recommender Systems, in Computer, vol. 42, no. 8, pp. 30-37, Aug. 2009, doi: 10.1109/MC.2009.263.

BellKor's Progress



Putting It All Together

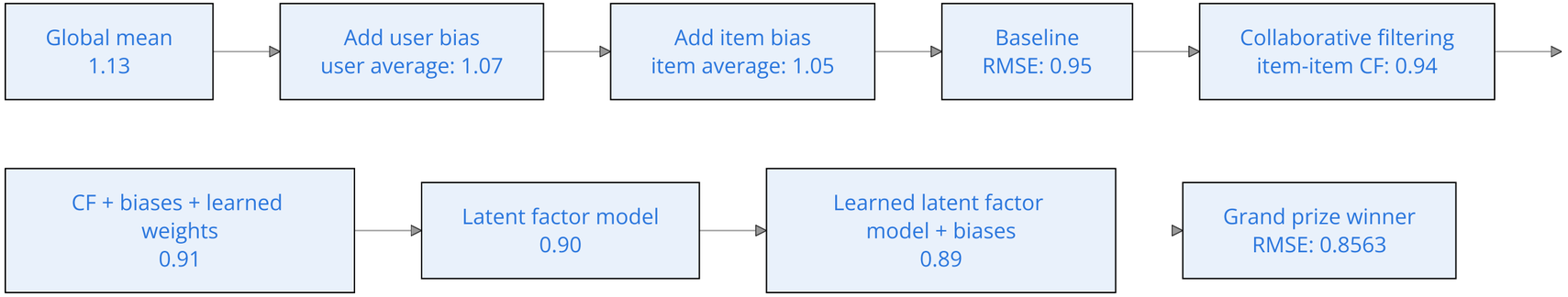
$$\hat{r}_{x,i} = \mu + b_x + b_i + p_x^T q_i$$

solved via (stochastic) gradient descent, with regularization to prevent overfitting, and the following loss function:

$$\min_{Q,P} \sum_{(x,i)\in R} \left(r_{xi} - \left(\mu + b_x + b_i + q_i^\top p_x\right)\right)^2 +$$

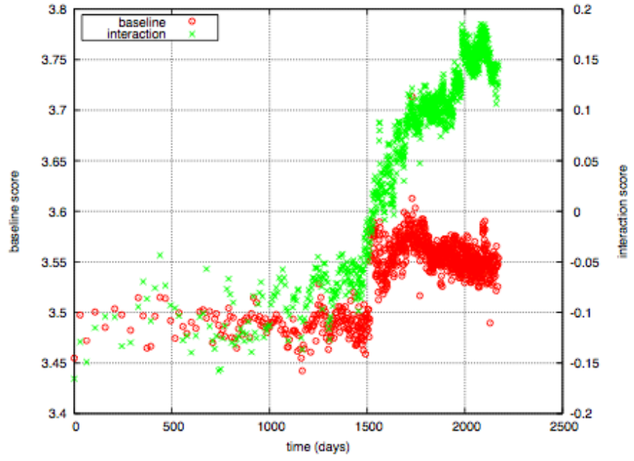
$$\left(\lambda_1 \sum_i \|q_i\|^2 + \lambda_2 \sum_x \|p_x\|^2 + \lambda_3 \sum_x |b_x|^2 + \lambda_4 \sum_i |b_i|^2\right)$$

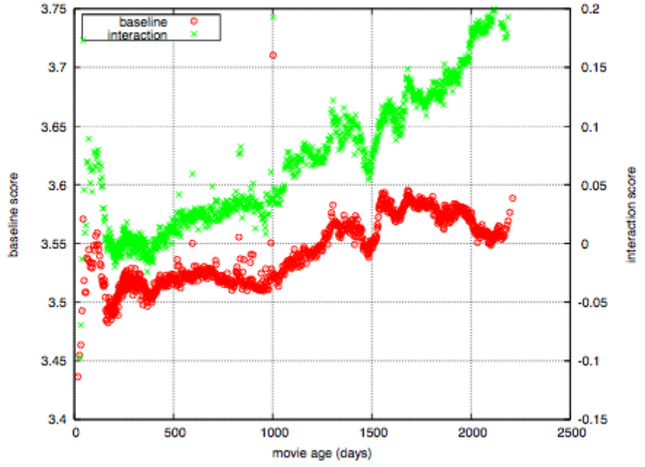
BellKor's Progress



Temporal Biases Of Users

- Sudden rise in the average movie rating (early 2004)
  - Improvements in Netflix
  - GUI improvements
  - Meaning of rating changed
- Movie age
  - Users prefer new movies without any reasons
  - Older movies are just inherently better than newer ones



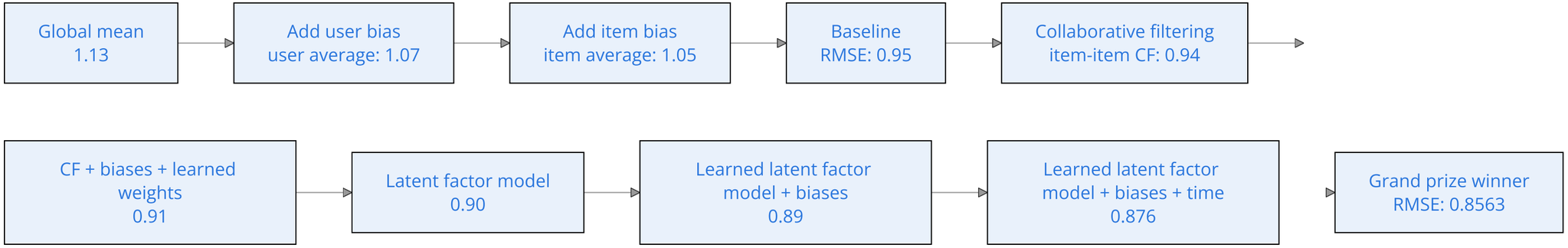


Temporal Biases & Factors

- Original model: $\hat{r}_{x,i} = \mu + b_x + b_i + p_x^T q_i$
- Add time dependence to biases: $\hat{r}_{x,i}(t) = \mu + b_x(t) + b_i(t) + p_x^T q_i$
  - Options:
    - 1. Parameterize time-dependence by linear trends
    - 2. Each bin corresponds to 10 consecutive weeks: $b_i(t) = b_i + b_{i,Bin(t)}$
- Add temporal dependence to factors: $p_x(t)$ for user preference vector on day t

Yehuda Koren. 2009. Collaborative filtering with temporal dynamics. In Proceedings of the 15th ACM SIGKDD international conference on Knowledge discovery and data mining (KDD '09). Association for Computing Machinery, New York, NY, USA, 447–456. https://doi.org/10.1145/1557019.1557072

BellKor's Progress



Standing on June 26th, 2009

NETFLIX

Netflix Prize

COMPLETED

Home | Rules | Leaderboard | Update | Download

Leaderboard

Showing Test Score. Click here to show quiz score

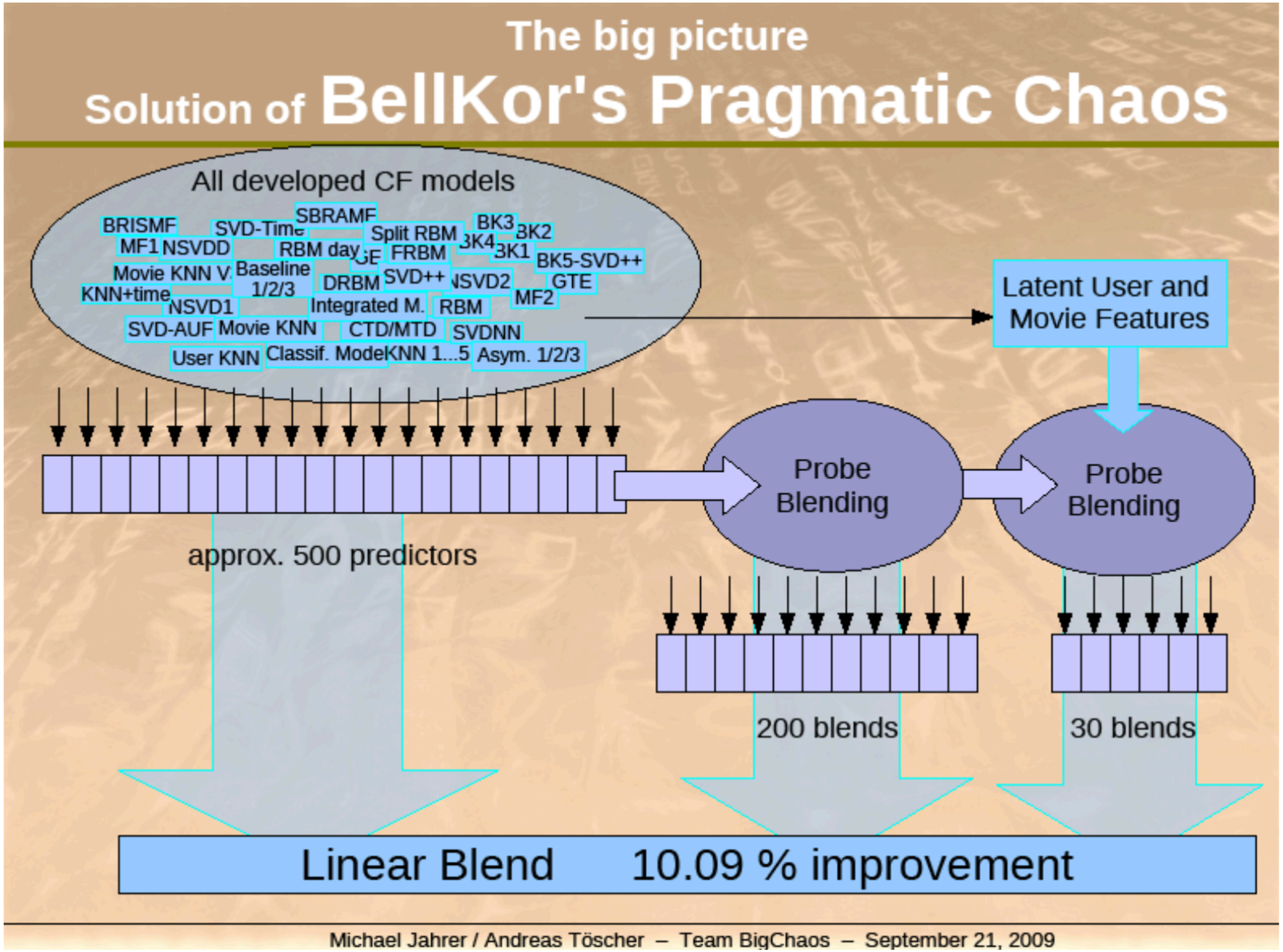
Display top 20 leaders.

Rank	Team Name	Best Test Score	% Improvement	Best Submit Time
Grand Prize - RMSE = 0.8567 - Winning Team: BellKor's Pragmatic Chaos				
1	BellKor's Pragmatic Chaos	0.8567	10.06	2009-07-26 18:18:28
2	The Ensemble	0.8567	10.06	2009-07-26 18:38:22
3	Grand Prize Team	0.8582	9.90	2009-07-10 21:24:40
4	Opera Solutions and Vandelay United	0.8588	9.84	2009-07-10 01:12:31
5	Vandelay Industries !	0.8591	9.81	2009-07-10 00:32:20
6	PragmaticTheory	0.8594	9.77	2009-06-24 12:06:56
7	BellKor in BigChaos	0.8601	9.70	2009-05-13 08:14:09
8	Dace_	0.8612	9.59	2009-07-24 17:18:43
9	Feeds2	0.8622	9.48	2009-07-12 13:11:51
10	BigChaos	0.8623	9.47	2009-04-07 12:33:59
11	Opera Solutions	0.8623	9.47	2009-07-24 00:34:07
12	BellKor	0.8624	9.46	2009-07-26 17:19:11
Progress Prize 2008 - RMSE = 0.8627 - Winning Team: BellKor in BigChaos				
13	xiangliang	0.8642	9.27	2009-07-15 14:53:22
14	Gravity	0.8643	9.26	2009-04-22 18:31:32
15	Ces	0.8651	9.18	2009-06-21 19:24:53
16	Invisible Ideas	0.8653	9.15	2009-07-15 15:53:04
17	Just a guy in a garage	0.8662	9.06	2009-05-24 10:02:54
18	J Dennis Su	0.8666	9.02	2009-03-07 17:16:17
19	Craig Carmichael	0.8666	9.02	2009-07-25 16:00:54
20	acmehill	0.8668	9.00	2009-03-21 16:20:50
Progress Prize 2007 - RMSE = 0.8723 - Winning Team: KorBell				

The Last 30 Days

- Ensemble team formed
  - Group of other teams on leaderboard forms a new team.
  - Relies on combining their models.
  - Quickly also get a qualifying score over 10%.
- BellKor's Pragmatic Chaos
  - Continue to get small improvements in their scores.
  - Realize that they are in direct competition with Ensemble.
- Strategy
  - Both teams carefully monitoring the leaderboard.
  - Only sure way to check for improvement is to submit a set of predictions, but this alerts the other team of your latest score.

Ensemble Strategy


The big picture
Solution of BellKor's Pragmatic Chaos
All developed CF models
BRISMF
SVD-Time
SBRAMF
Split RBM
BK3
BK2
MF1
NSVDD
RBM day
FRBM
BK4
BK1
BK5-SVD++
Movie KNN V
Baseline 1/2/3
DRBM
SVD++
NSVD2
GTE
KNN+time
NSVD1
Integrated M.
RBM
MF2
SVD-AUF
Movie KNN
CTD/MTD
SVDNN
User KNN
Classif. Mode
KNN 1...5
Asym. 1/2/3
Latent User and Movie Features
Probe Blending
Probe Blending
approx. 500 predictors
200 blends
30 blends
Linear Blend 10.09 % improvement
Michael Jahrer / Andreas Töscher – Team BigChaos – September 21, 2009

24 Hours from the Deadline

- Submissions limited to 1 a day
  - Only 1 final submission could be made in the last 24h.
- 24 hours before deadline, BellKor team member in Austria notices (by chance) that Ensemble posts a score that is slightly better than BellKor's
- Frantic last 24 hours for both teams
  - Much computer time on final optimization
  - Carefully calibrated to end about an hour before deadline
- Final submissions
  - BellKor submits a little early (on purpose), 40 mins before deadline
  - Ensemble submits their final entry 20 mins later
  - 40,000 teams in total

Final Results

NETFLIX

Netflix Prize

COMPLETED

Home | Rules | Leaderboard | Update | Download

Leaderboard

Showing Test Score. Click here to show quiz score

Display top 20 leaders.

Rank	Team Name	Best Test Score	% Improvement	Best Submit Time
Grand Prize - RMSE = 0.8567 - Winning Team: BellKor's Pragmatic Chaos				
1	BellKor's Pragmatic Chaos	0.8567	10.06	2009-07-26 18:18:28
2	The Ensemble	0.8567	10.06	2009-07-26 18:38:22
3	Grand Prize Team	0.8582	9.90	2009-07-10 21:24:40
4	Opera Solutions and Vandelay United	0.8588	9.84	2009-07-10 01:12:31
5	Vandelay Industries !	0.8591	9.81	2009-07-10 00:32:20
6	PragmaticTheory	0.8594	9.77	2009-06-24 12:06:56
7	BellKor in BigChaos	0.8601	9.70	2009-05-13 08:14:09
8	Dace_	0.8612	9.59	2009-07-24 17:18:43
9	Feeds2	0.8622	9.48	2009-07-12 13:11:51
10	BigChaos	0.8623	9.47	2009-04-07 12:33:59
11	Opera Solutions	0.8623	9.47	2009-07-24 00:34:07
12	BellKor	0.8624	9.46	2009-07-26 17:19:11
Progress Prize 2008 - RMSE = 0.8627 - Winning Team: BellKor in BigChaos				
13	xiangliang	0.8642	9.27	2009-07-15 14:53:22
14	Gravity	0.8643	9.26	2009-04-22 18:31:32
15	Ces	0.8651	9.18	2009-06-21 19:24:53
16	Invisible Ideas	0.8653	9.15	2009-07-15 15:53:04
17	Just a guy in a garage	0.8662	9.06	2009-05-24 10:02:54
18	J Dennis Su	0.8666	9.02	2009-03-07 17:16:17
19	Craig Carmichael	0.8666	9.02	2009-07-25 16:00:54
20	acmehill	0.8668	9.00	2009-03-21 16:20:50
Progress Prize 2007 - RMSE = 0.8723 - Winning Team: KorBell				

Final Results